

Token usage and limits service

front: what to ask, what to design · a gate in front of a meter · never block the model by accident

Anthropic questions this is: not a card on its own (Tyler's opener). It is the **gateway box of SD Q1 inference API** (per-key and per-account quotas, admission, tiering; "no rate limiting" is a planted flaw in the doc-review round, and Aug 2026 reports want load shedding + rate limiting covered) · **igotanooffer Q5** developer APIs to access models (keys, **quotas**, versioning, streaming, **usage tracking**) · **prachub #12** distributed rate limiter with fault tolerance (Rik's Q87) · **hellointerview** prompt playground platform flavour (per-provider token buckets). **Where Anthropic asks it:** as the gateway of Q1 and as "tighten limits dynamically on queue depth" under overload.

1 REQUIREMENTS 0–5 min · scale in one sentence, window in minute 3

- **Ask:** caller = person or employee under a company, can one be both? window: rolling 5 h, per minute, monthly? tokens, dollars, or both? input and output weighted the same? hard cap or soft warning? scale: every model request, tens of thousands/s.
- **A caller can:** ask "may this account spend N more tokens?" → yes/no in single-digit ms · admin sets a limit for a person or an org · org admin sets per-member limits inside the org cap · the system **records** tokens actually used · read usage and remaining.
- **Technical challenges:** check < 5 ms P99 (in front of every request) · availability above the model API's own · **availability over consistency** (racing requests may both pass; we are not a bank) · never block the model by accident · exact counts exist only after decoding → check is an estimate, record trues it up.

2 CORE ENTITIES 5–9 min · five, with fields

- **User**(id, email) · **Organisation**(id, name) · **Membership**(user_id, org_id, role); personal account = user with no membership
- **Limit**(scope user|org, scope_id, window type, max_tokens, max_dollars?, set_by, set_at)
- **Usage**(scope, scope_id, window_start, tokens_in, tokens_out, dollars) — one row per scope per window
- **Hierarchy:** an employee check = member counter **and** org counter; member limits are over-allocated on purpose (20 engineers × \$100 under a \$1,000 cap).
- Say the link (user_id), no arrows between tables; no pseudocode in this round.

3 API 9–13 min · four endpoints, the fourth is the clock

- GET /usage?scope=user|org&id= → limit, used, window start/end, remaining
- PUT /limits {scope, id, window, max} — org admins only inside their org, under its cap
- POST /usage/check {scope_chain, estimated_tokens} → allow | deny + retry_after
- POST /usage/record {request_id, scope_chain, tokens_in, tokens_out} — idempotent on request_id. **The one missed in the mock:** a check without a record is a gate with no clock.

4 DATA FLOW 13–20 min · check before the GPU, record after

- client → model API gateway → **check** (input length + guessed output) → usage service reads member + org counters and limits from Redis → allow (forward to the inference pool) or **429** with window end →
- GPU finishes → the inference server (only place with true counts) emits a usage event {request_id, scopes, in, out} → queue → consumer dedupes on request_id, **INCRBY** both counters, appends to the ledger →
- nightly reconcile compares ledger sums with counters; billing reads the ledger, never the counters.

5 HIGH-LEVEL DESIGN 20–33 min · gate above, meter below

- Top: clients → gateway → usage service with Redis counters (usage: {scope}: {id}: {window_start}, TTL a little over the window) and Postgres as source of truth for limits → GPU pool.
- Bottom, dashed: reporter beside the inference server → topic → consumer → ledger → reconcile.
- **Numbers:** one Redis node > 100k ops/s; shard by scope id when it does not · limits cached with a long TTL · rolling 5 h as a fixed bucket keyed by window start; true sliding window (sorted set) only for per-minute API limits.
- Solid arrows = synchronous request, numbered; dashed = async record.

6 DEEP DIVES 33–48 min · fail open, windows, reconcile

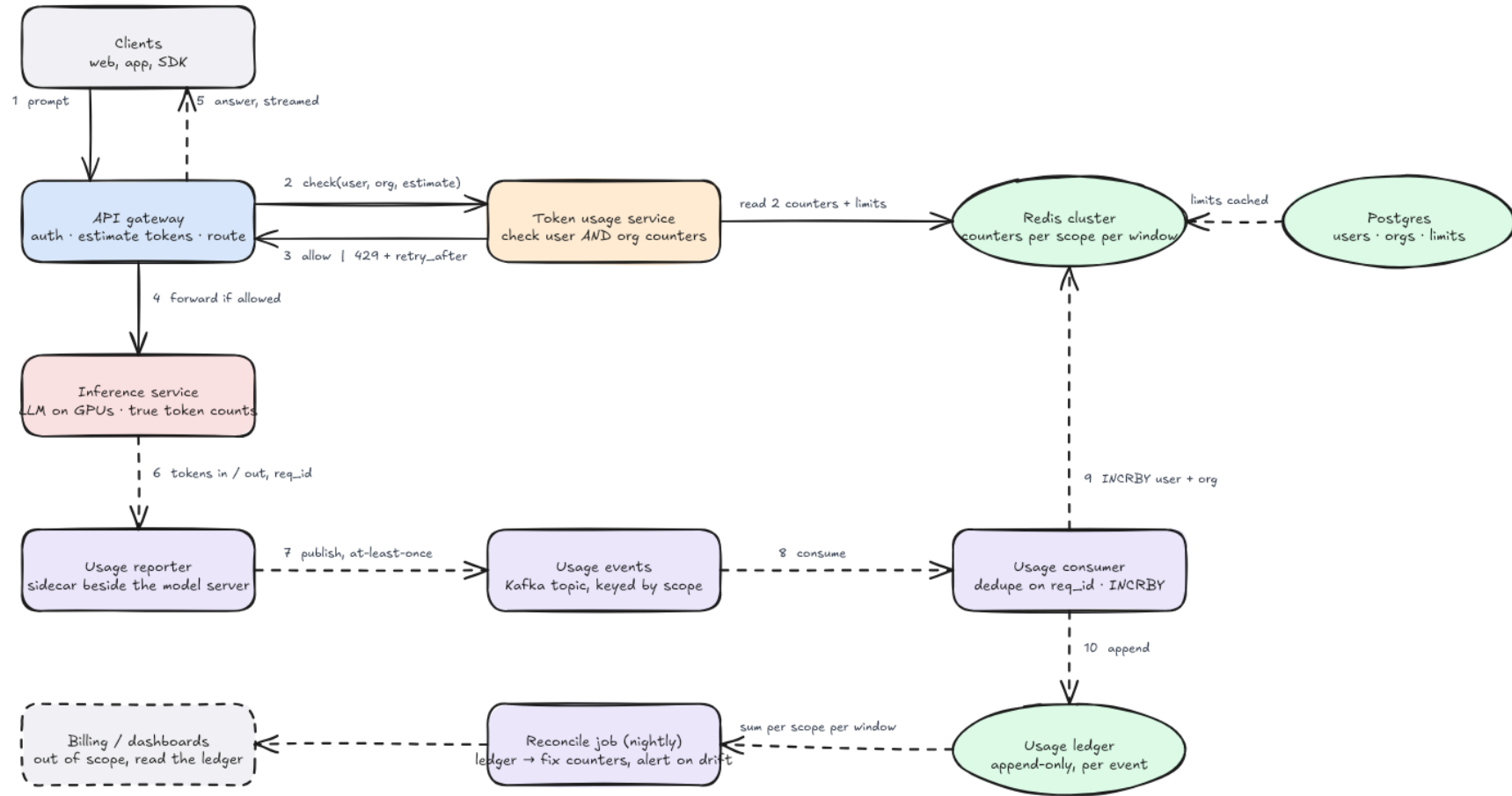
- **Fail open, on purpose:** Redis timeout → allow + log a miss; queue backed up → usage late, not lost. Exception: a contractually hard org cap fails closed, and the contract says so.
- **At-least-once record:** request_id on every event, short dedupe set, ledger is the truth.
- **Estimate on check:** from max_tokens or history; over-spend bounded by one request; true up on record.
- **Reconcile:** nightly ledger sums vs counters, fix drift, alert on its size.
- **Tie-in to SD Q1:** the same counters let the gateway tighten limits on queue depth and shed free tier first under overload.
- **Bow:** a synchronous gate in front of an asynchronous meter; two counters per scope; an employee is two checks; check before the GPU, record after; fail open unless contractually hard; bill from the ledger.

The board at minute forty

back: the corrected design, as it should look on the shared screen

Token usage and limits service

System diagram. Gate before the GPU, meter after it, ledger behind. Solid = synchronous request path, dashed = asynchronous or background.



Legend

- client / out of scope
- gateway
- the service under design
- storage
- GPU inference
- asynchronous path

The request, in ten steps

- 1 client sends a prompt
- 2 gateway asks: may user u in org o spend about N tokens?
- 3 service reads two counters, answers allow or 429 with retry_after
- 4 gateway forwards to the GPUs
- 5 answer streams back to the client
- 6 inference server emits the true input and output token counts
- 7 reporter publishes the event
- 8 consumer reads it, at-least-once, dedupes on req_id
- 9 counters incremented for user and org
- 10 event appended to the ledger; the nightly job sums the ledger, fixes counter drift, and billing reads the ledger, never the counters